

WEEK 6: PRIM'S ALGORITHM

Advanced Algorithms

ROADMAP

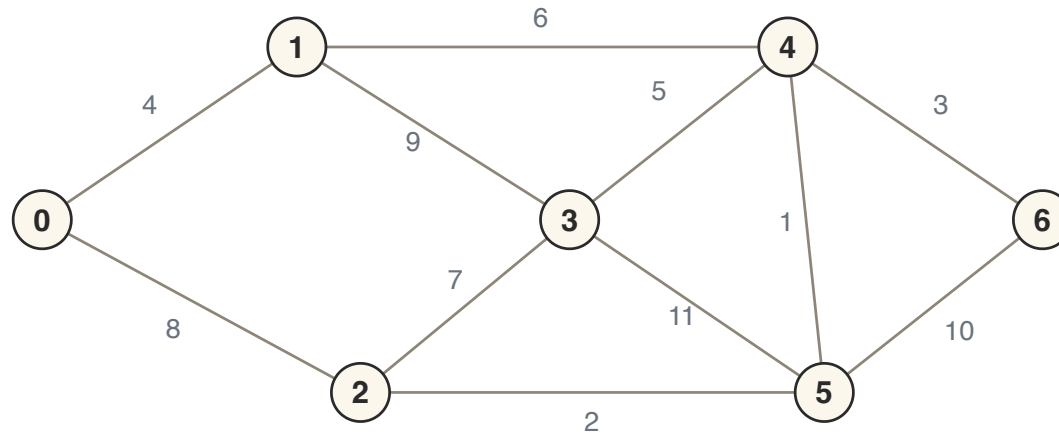
1. Dijkstra review
2. Bottleneck (minimax) distance
3. Prim's algorithm
4. Borůvka's algorithm

One idea powers everything today: the MST contains a minimum-weight edge of every cut (Week 5).

DIJKSTRA'S ALGORITHM (REVIEW)

SINGLE-SOURCE SHORTEST PATHS

Dijkstra's algorithm solves the single-source shortest path problem in a weighted (directed or undirected) graph with **non-negative** weights.

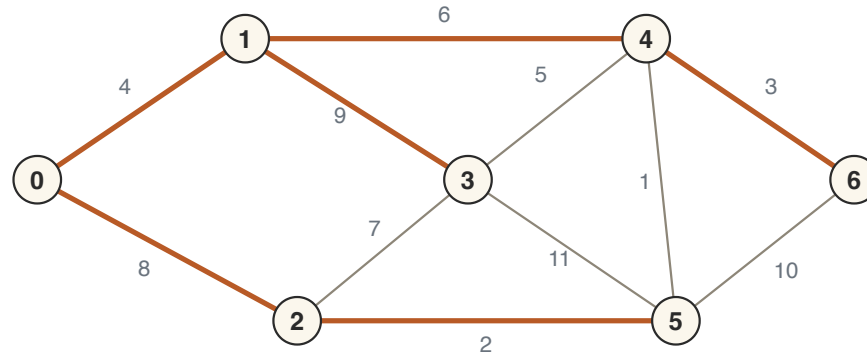


Our running example. Source vertex: 0.

What is the shortest path from the source to *every other* vertex?

THE SHORTEST-PATH TREE

The shortest paths from a source to every other vertex can be represented by a *tree*.



The shortest-path tree from vertex 0 (highlighted).

Question: is this a minimum spanning tree?

No! This tree has weight 32; the MST has weight 21. But the two problems are close cousins, as we will see today.

DIJKSTRA RECAP

Initialise an array `dist_to` with 0 for the source and ∞ for every other vertex.

Repeat: pick the unprocessed vertex u with smallest `dist_to` value, *relax* all its outgoing edges, and mark u as processed.

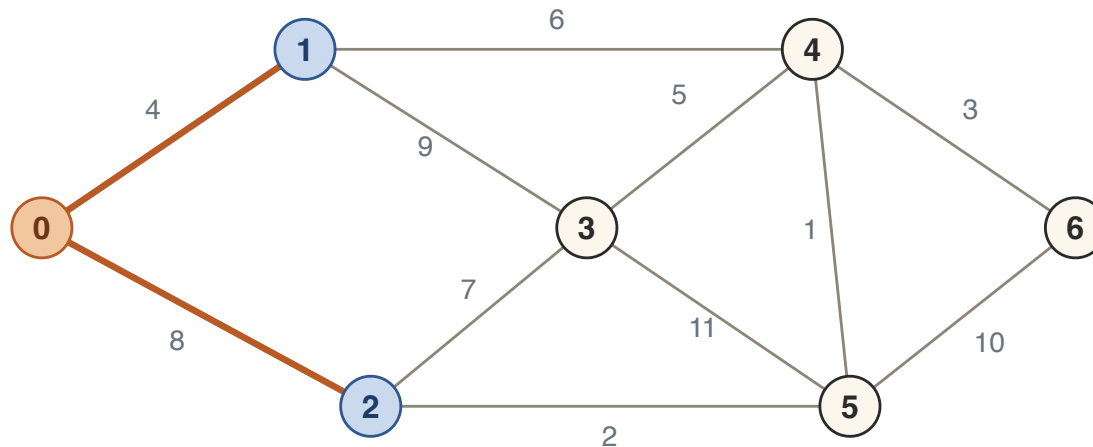
RELAX EDGE (u, v)

$$\text{dist_to}[v] = \min(\text{dist_to}[v], \text{dist_to}[u] + w(u, v))$$

Colour code for the pictures that follow: processed in the frontier (finite dist)
being processed

DIJKSTRA: A WORKED RUN

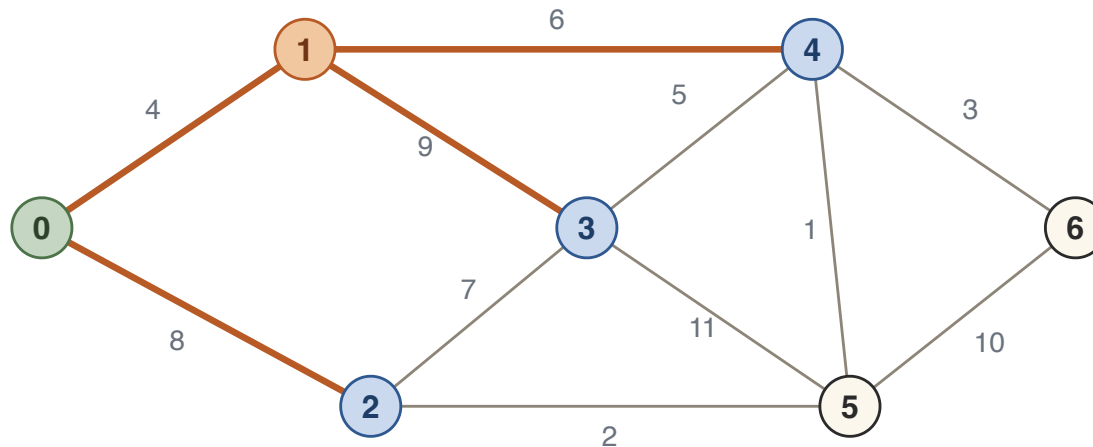
Start: `dist_to[0]` = 0, everything else ∞ . Process 0: relax 0-1 ($0 + 4 = 4$) and 0-2 ($0 + 8 = 8$).



dist_to						
0	4	8	∞	∞	∞	∞
0	1	2	3	4	5	6

DIJKSTRA: A WORKED RUN

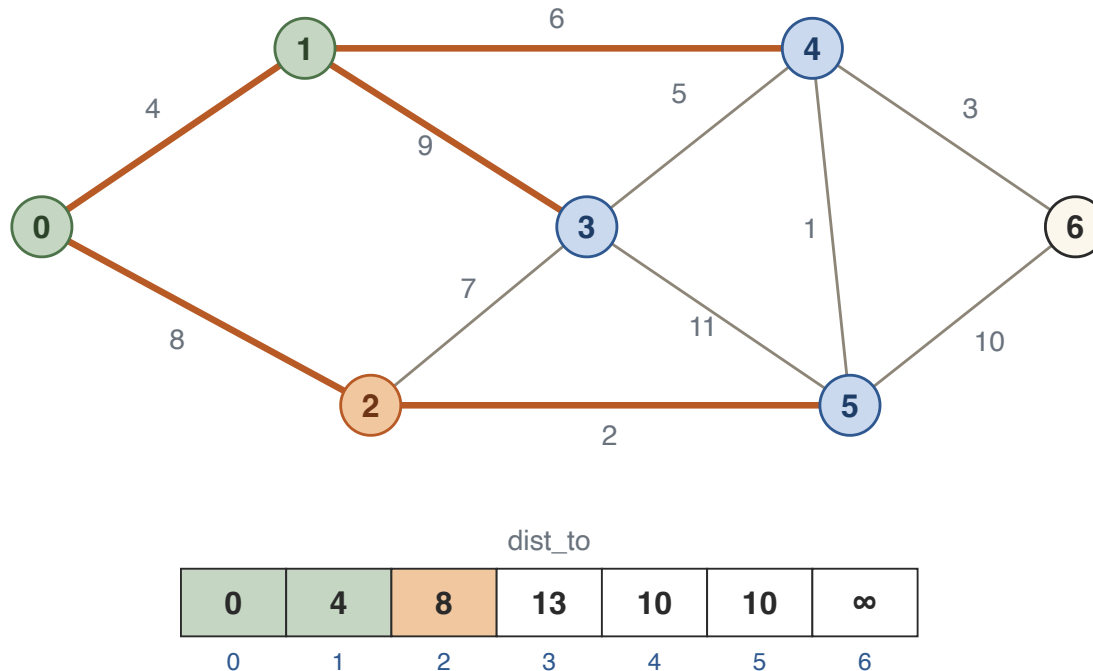
Smallest unprocessed `dist_to` is 1 (at 4). Process 1: relax 1-3 ($4 + 9 = 13$) and 1-4 ($4 + 6 = 10$).



dist_to						
0	4	8	13	10	∞	∞
0	1	2	3	4	5	6

DIJKSTRA: A WORKED RUN

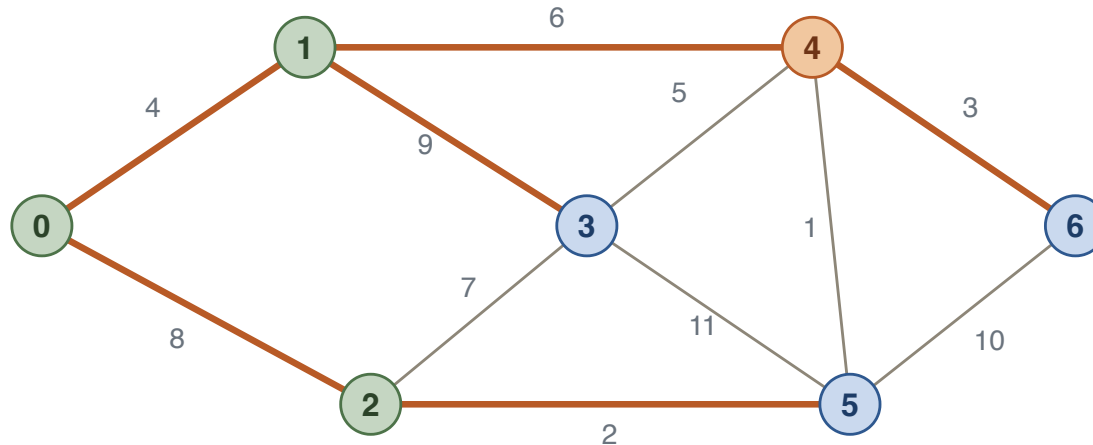
Next is 2 (at 8). Relax 2-5 ($8 + 2 = 10$); 2-3 gives $8 + 7 = 15$, no better than 13, so nothing changes there.



Vertices 4 and 5 now tie at 10 — we may break the tie arbitrarily. Take 4.

DIJKSTRA: A WORKED RUN

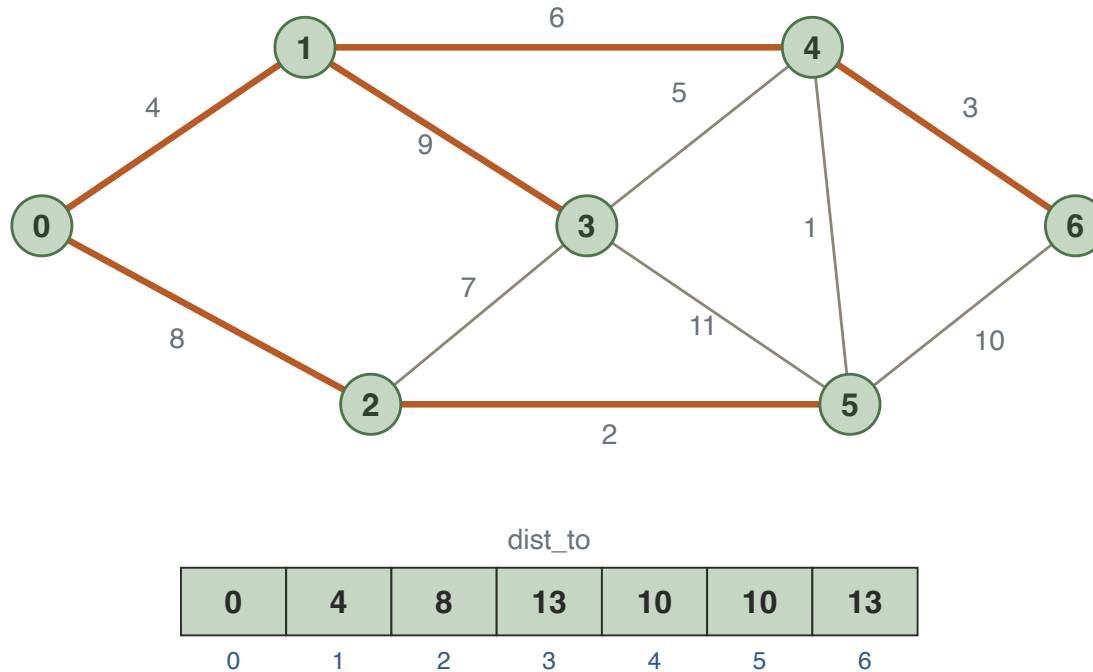
Process 4 (at 10). Relax 4-6 ($10 + 3 = 13$); 4-3 (15) and 4-5 (11) do not improve.



dist_to						
0	4	8	13	10	10	13
0	1	2	3	4	5	6

DIJKSTRA: A WORKED RUN

Then 5, 3 and 6 in turn: nothing improves, and the run ends.



Processing order: 0, 1, 2, 4, 5, 3, 6 with distances 0, 4, 8, 10, 10, 13, 13 — **nondecreasing**. Keep this in mind.

KEY DIJKSTRA CLAIM

CLAIM If the graph has non-negative weights, then when a vertex is processed its `dist_to` value equals its actual distance from the source.

We show this by induction on the number of vertices processed.

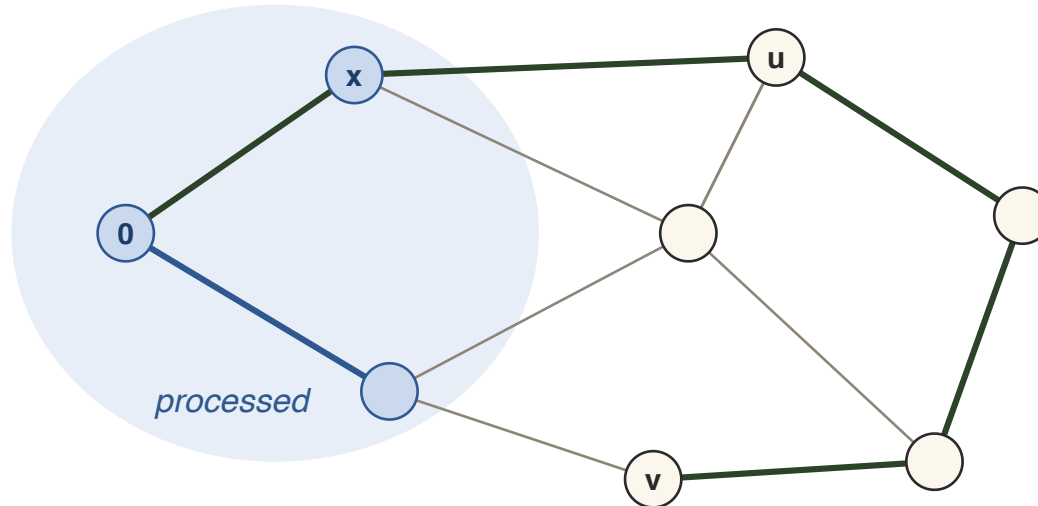
Base case: the first vertex processed is the source itself, with `dist_to` = 0. The actual distance to the source is 0, so the claim holds.

KEY DIJKSTRA CLAIM

Inductive step: suppose the claim holds for all processed vertices, and let v be the unprocessed vertex with smallest `dist_to` — the next to be processed.

We must show $d(0, v) = \text{dist_to}[v]$ — the proof sketch is on the next slides.

KEY CLAIM: PROOF SKETCH



A hypothetical shorter path (green) to v ; u is its first unprocessed vertex, reached from processed x .

Suppose for contradiction some path from the source to v is shorter than $\text{dist_to}[v]$. Let u be the *first unprocessed* vertex on that path.

KEY CLAIM: PROOF SKETCH

By the inductive hypothesis, $d(0, x) = \text{dist_to}[x]$ held when we relaxed the edge (x, u) , so

$\text{dist_to}[u] \leq d(0, x) + w(x, u) = d(0, u)$, hence

$\text{dist_to}[u] = d(0, u)$. Then:

$$d(0, v) = d(0, u) + d(u, v)$$

$$\geq d(0, u)$$

weights are non-negative

$$= \text{dist_to}[u]$$

$$\geq \text{dist_to}[v]$$

v is processed next

KEY CLAIM: PROOF SKETCH

So $d(0, v) \geq \text{dist_to}[v]$. We always have $d(0, v) \leq \text{dist_to}[v]$ (relaxations only record real path lengths), hence $d(0, v) = \text{dist_to}[v]$. ■

CAREFUL Non-negativity is used exactly once, in $d(u, v) \geq 0$: the unprocessed tail of the green path can't shorten it.

SORTING PROPERTY

Say the order of vertices processed by Dijkstra's algorithm is $0 = v_1, v_2, \dots, v_n$.

FACT $d(0, v_2) \leq d(0, v_3) \leq \dots \leq d(0, v_n)$. Dijkstra outputs distances in sorted order.

SORTING PROPERTY

Why? Consider v_i and v_{i+1} . Two options:

- **dist_to** $[v_{i+1}]$ *did not change* when v_i was processed: then $d(0, v_i) \leq \mathbf{dist_to}[v_{i+1}] = d(0, v_{i+1})$, since v_i was chosen first.
- **dist_to** $[v_{i+1}]$ *decreased* when v_i was processed: then the shortest path to v_{i+1} goes through v_i , so $d(0, v_i) \leq d(0, v_{i+1})$ because weights are non-negative.

THE SORTING BARRIER

Dijkstra's algorithm is *optimal* among algorithms that must produce the distances in sorted order: it cannot beat the cost of sorting.

But sorted output is not required by the shortest-path problem itself!

THE SORTING BARRIER

RECENT RESULT Duan, Mao, Mao, Shu, Yin (2025): a deterministic $O(|E| \log^{2/3} n)$ -time algorithm for single-source shortest paths with non-negative weights — the first to beat Dijkstra's $O(|E| + n \log n)$ bound on sparse graphs.

"Breaking the Sorting Barrier for Directed Single-Source Shortest Paths".

BOTTLENECK (MINIMAX) DISTANCE

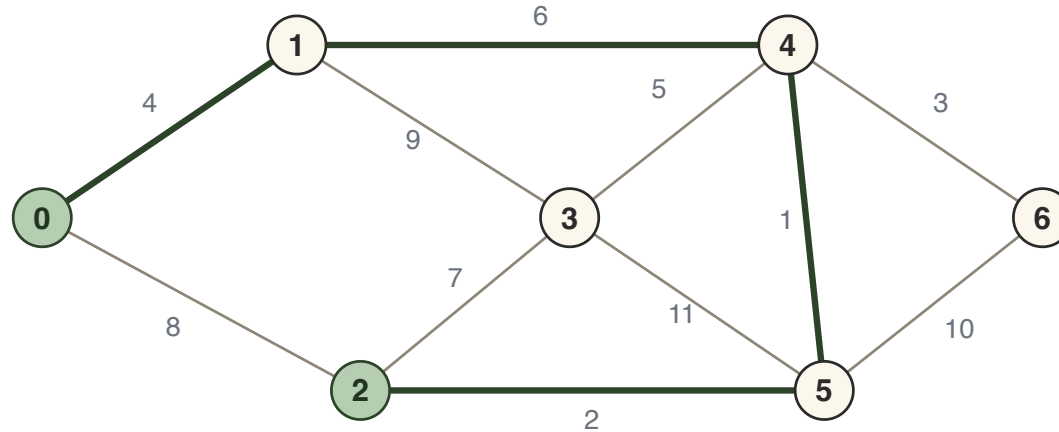
LONGEST-LEG DISTANCE

DEFINITION The **longest-leg distance** between vertices u and v is the minimum, over all paths between them, of the *maximum edge weight* on the path.

Think of a hike broken into legs: you want the route whose single hardest leg is as easy as possible.

Also known as the **minimax distance** or **bottleneck distance**.

EXAMPLE



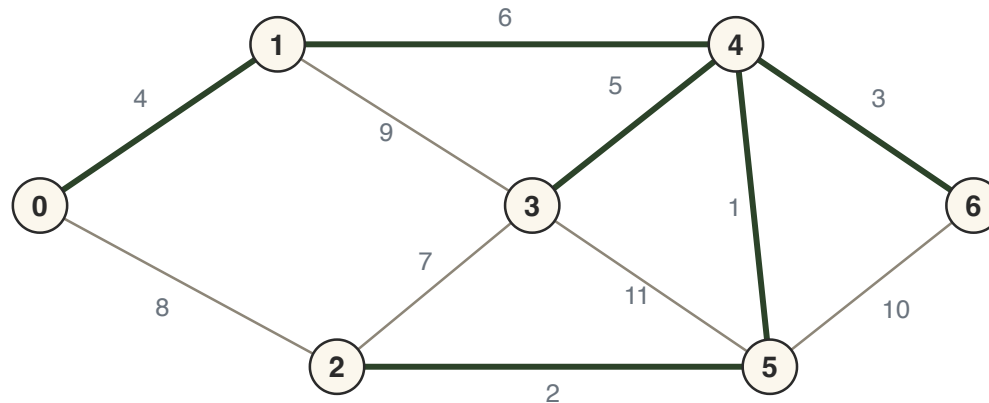
The green path 0, 1, 4, 5, 2 has legs 4, 6, 1, 2 — longest leg 6.

The direct edge $\{0, 2\}$ has weight 8. The green detour never uses a leg heavier than 6.

In this graph the longest-leg distance between 0 and 2 is **6**.

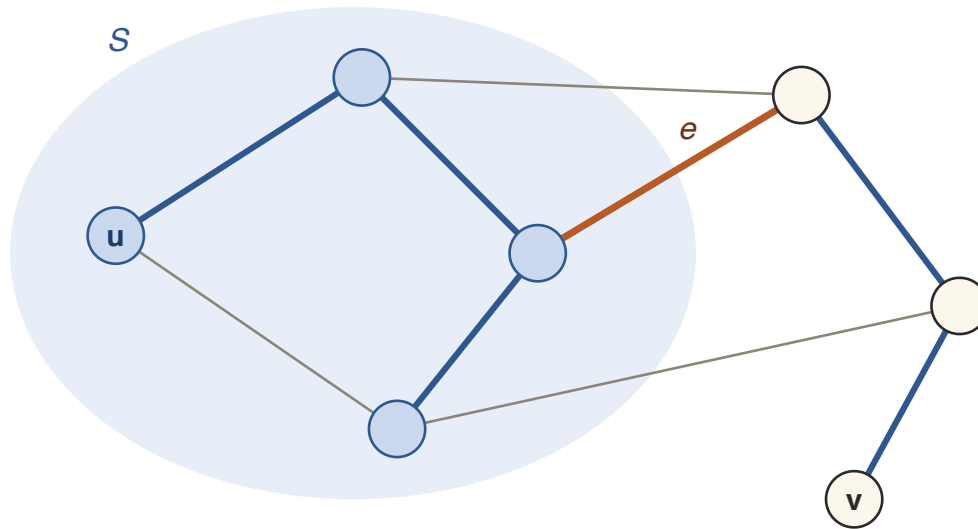
THE MST SOLVES MINIMAX PATHS

CLAIM A minimum spanning tree gives a shortest longest-leg path between *every* pair of vertices.



The MST of the running example (weight 21). The unique MST path 0, 1, 4, 5, 2 realises the bottleneck distance 6.

PROOF: SET UP THE CUT



Blue: MST edges. Removing e splits the tree; S = the side containing u .

Suppose for contradiction the MST path between u and v is *not* a shortest longest-leg path. Let e be the heaviest edge on this MST path. Remove e from the tree, and let S be the vertices still connected to u .

PROOF: EXCHANGE ACROSS THE CUT

A better path from u to v starts inside S and ends outside, so it must use some edge f in the cut defined by S .

Since the better path's longest leg beats the MST path's longest leg e , every edge on it — in particular f — satisfies $w(f) < w(e)$.

But then removing e from the tree and adding f reconnects it into a spanning tree of *smaller* total weight — contradicting minimality. ■

TAKEAWAY One tree, $n - 1$ edges, answers all $\binom{n}{2}$ minimax path queries at once.

DIJKSTRA FOR LONGEST-LEG PATHS

Can we adapt Dijkstra to compute longest-leg shortest paths (LLSP) from a source?

Only *relax* changes: path length is now the max leg, so extending by edge (u, v) costs `max` instead of `+`:

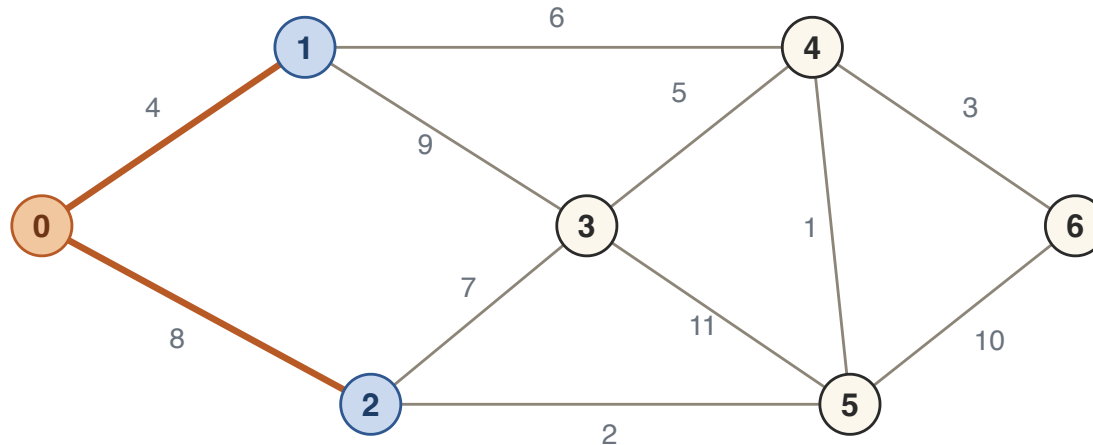
RELAX EDGE (u, v)

$$\text{dist_to}[v] = \min(\text{dist_to}[v], \max(\text{dist_to}[u], w(u, v)))$$

Everything else is identical: process the unprocessed vertex with smallest `dist_to`, relax its edges, repeat.

DIJKSTRA FOR LLSP: WORKED RUN

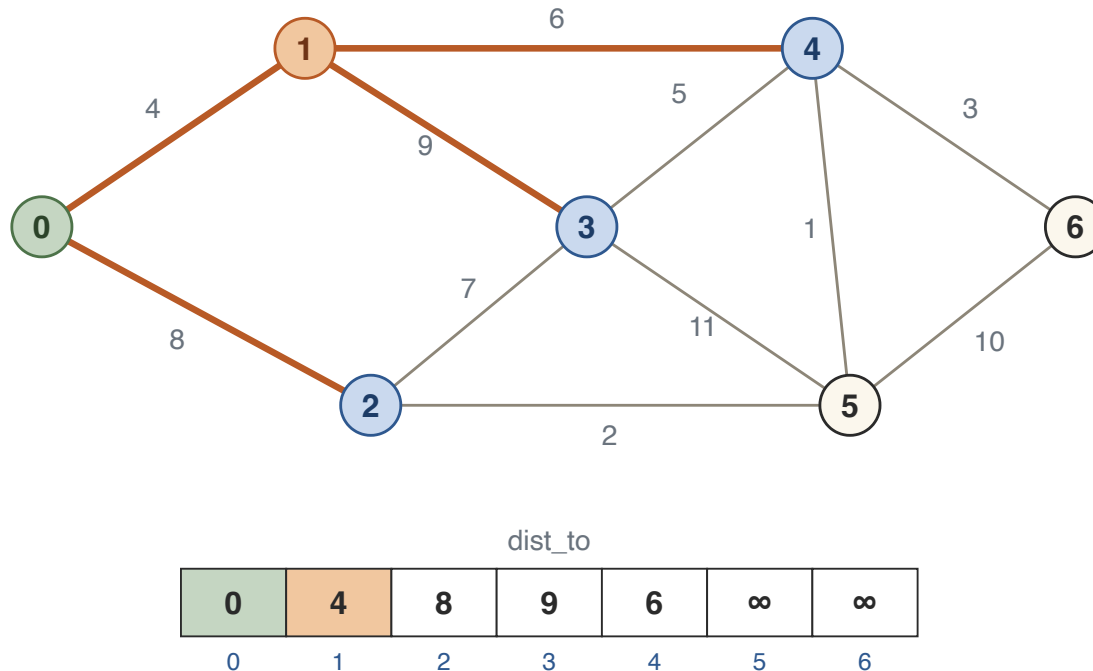
Source 0, same graph. Process 0: relax 0-1 ($\max(0, 4) = 4$) and 0-2 ($\max(0, 8) = 8$).



dist_to						
0	4	8	∞	∞	∞	∞
0	1	2	3	4	5	6

DIJKSTRA FOR LLSP: WORKED RUN

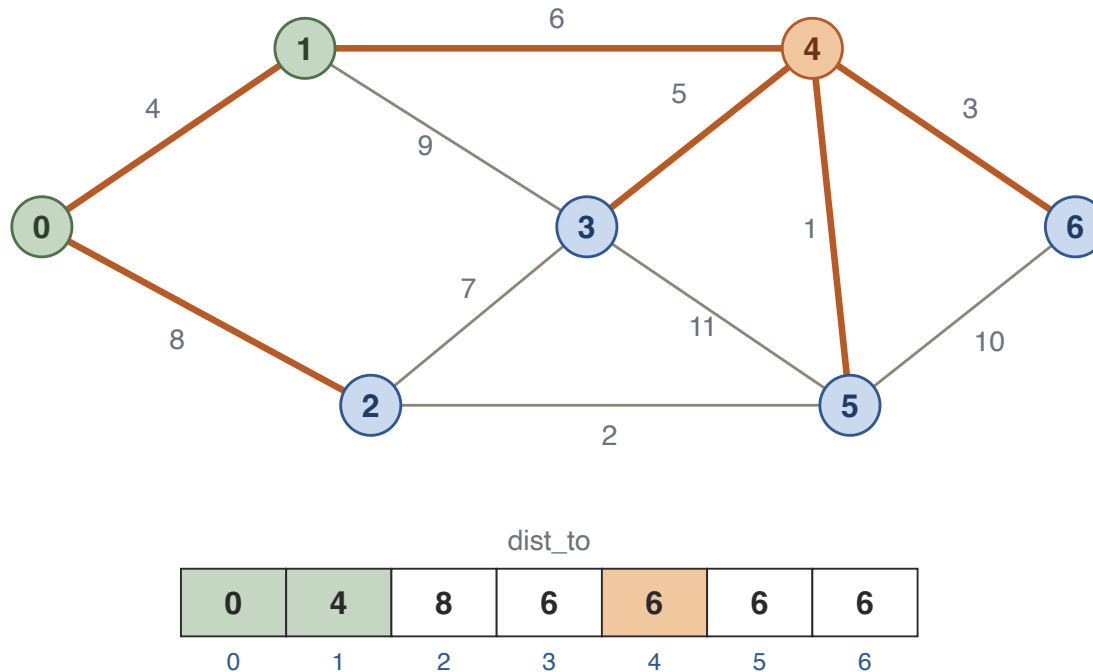
Process 1 (at 4): relax 1-3 ($\max(4, 9) = 9$) and 1-4 ($\max(4, 6) = 6$).



The smallest unprocessed value is now 6 at vertex 4 — it is processed next, *before* vertex 2 at 8.

DIJKSTRA FOR LLSP: WORKED RUN

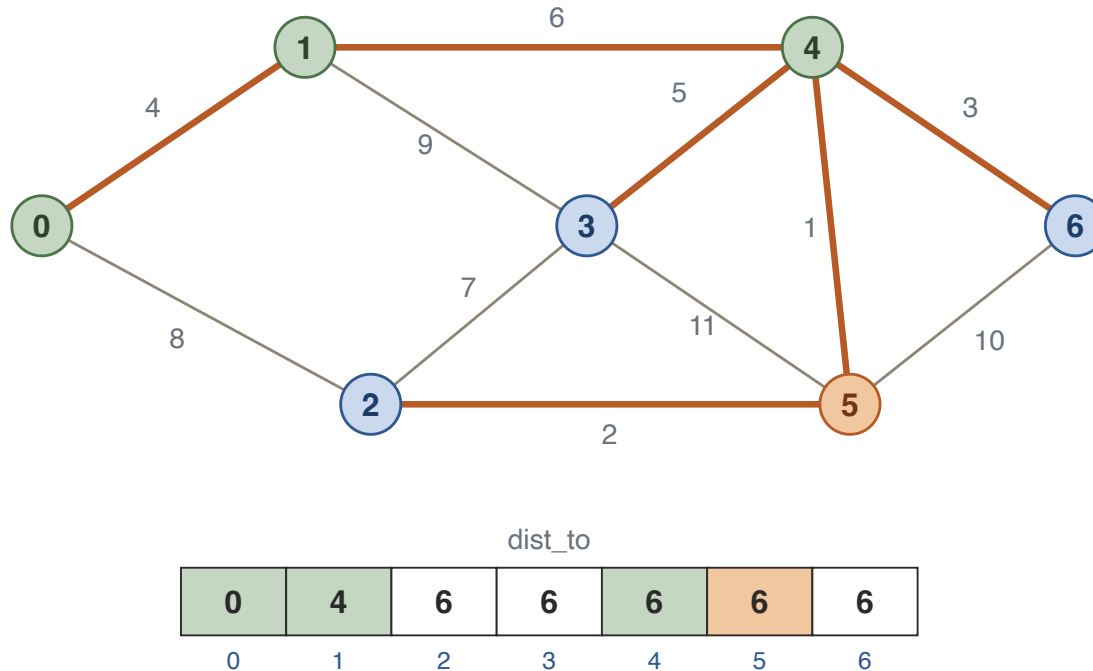
Process 4 (at 6): 4-3 ($\max(6, 5) = 6$) improves 3 from 9; 4-5 and 4-6 give 6 as well.



Vertices 3, 5 and 6 tie at 6 — take 5.

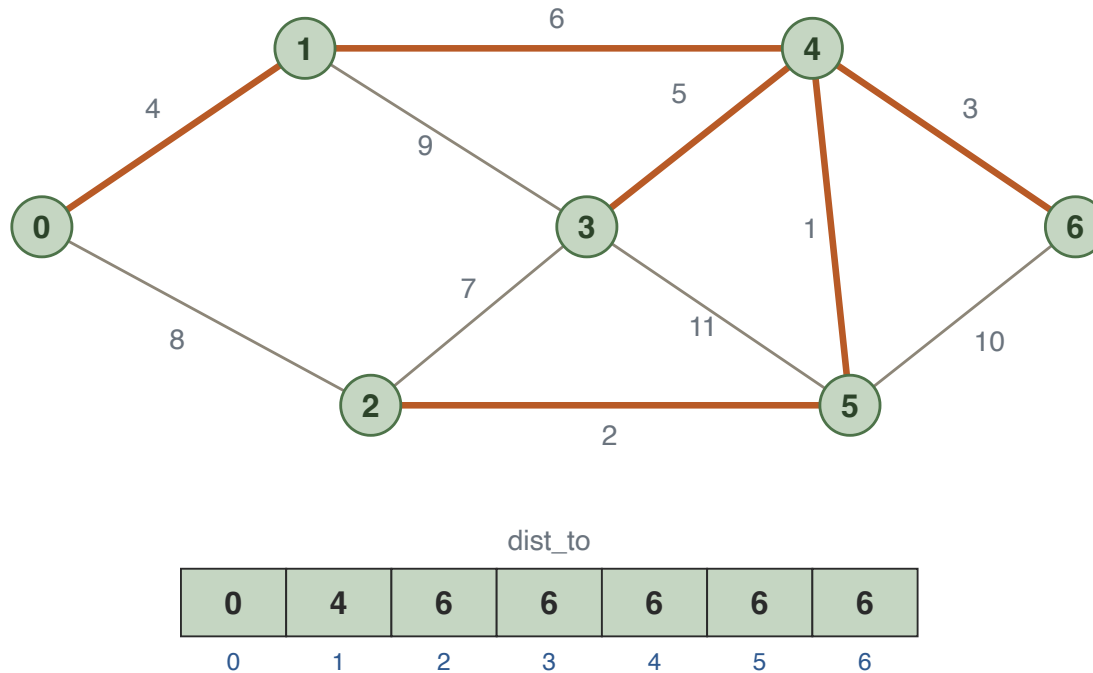
DIJKSTRA FOR LLSP: WORKED RUN

Process 5 (at 6): 5-2 ($\max(6, 2) = 6$) improves 2 from 8; 5-3 (11) and 5-6 (10) do not.



DIJKSTRA FOR LLSP: WORKED RUN

Then 3, 6 and 2 in turn: nothing improves, and the run ends.



Look at the highlighted tree of relaxations that stuck — it is exactly the **MST** from before!

LLSP DIJKSTRA: CORRECTNESS

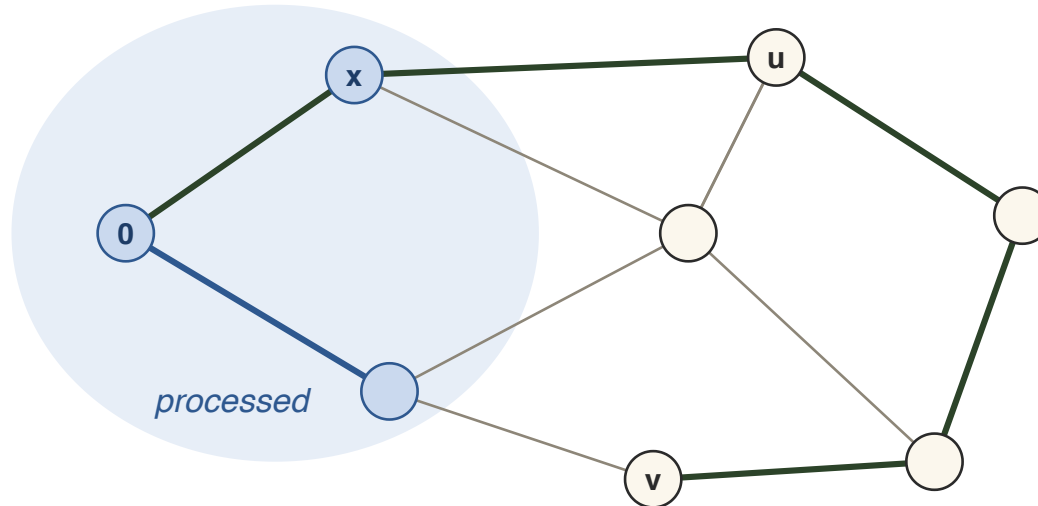
Same induction as before: *when a vertex is processed, its `dist_to` value is its longest-leg distance from the source.*

Let v be the next vertex to be processed: unprocessed, with smallest `dist_to`. Every finite `dist_to` is the longest leg of some real path, so $d(0, v) \leq \text{dist_to}[v]$ always.

ASSUMPTION (FOR CONTRADICTION)

$d(0, v) < \text{dist_to}[v]$: some path to v has longest leg $d(0, v)$, strictly smaller than `dist_to`[v].

LLSP DIJKSTRA: CORRECTNESS



A path to v with longest leg $d(0, v)$ (green); u is its first unprocessed vertex, reached from processed x .

Every leg of the green path is at most $d(0, v)$ — in particular $w(x, u) \leq d(0, v)$. And its prefix up to x is a path to x , so $d(0, x) \leq d(0, v)$ too.

LLSP DIJKSTRA: CORRECTNESS

When x was processed, edge (x, u) was relaxed, with $\text{dist_to}[x] = d(0, x)$ by the inductive hypothesis. So:

$$\begin{aligned}\text{dist_to}[u] &\leq \max(\text{dist_to}[x], w(x, u)) && \text{relaxing } (x, u) \\ &= \max(d(0, x), w(x, u)) && \text{inductive hypothesis} \\ &\leq d(0, v) && \text{previous slide} \\ &< \text{dist_to}[v] && \text{assumption}\end{aligned}$$

So the *unprocessed* vertex u has a strictly smaller dist_to than v — contradicting the choice of v . ■

PRIM'S ALGORITHM

THE CUT PRINCIPLE, AGAIN

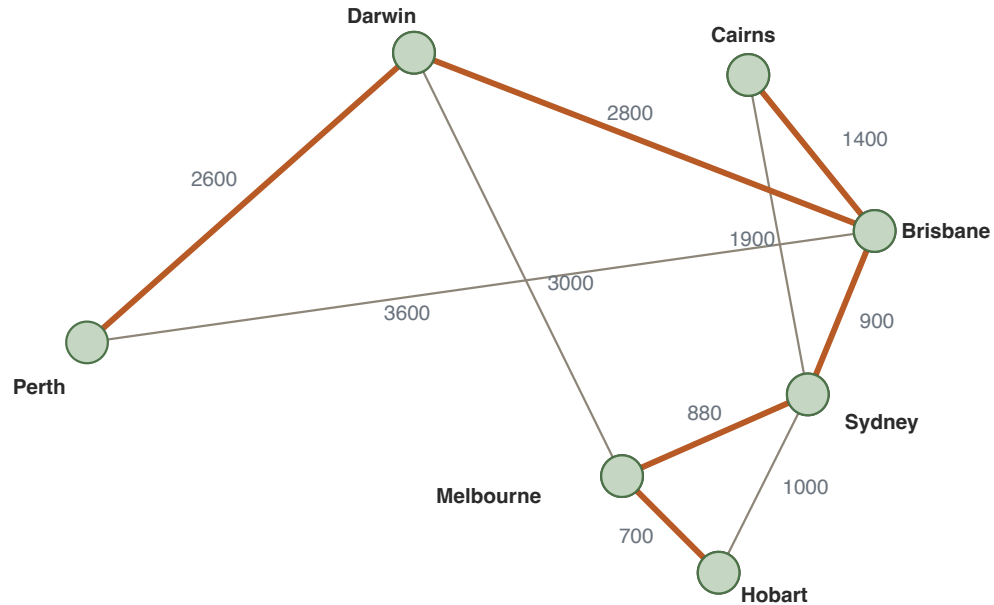
FACT (WEEK 5) A minimum spanning tree must contain a minimum-weight edge from every cut.

Kruskal used this fact with the cuts around growing forest components.

Prim's idea: grow a *single* tree from a starting vertex, always adding the minimum-weight edge in the cut around the tree built so far.

Every step is certified by the Fact — so the final tree is an MST.

PRIM IN ACTION



Flight-distance graph on Australian cities. Start the tree in Perth.

Each step adds the minimum-weight edge leaving the current tree S : Perth–Darwin 2600, Darwin–Brisbane 2800, Brisbane–Sydney 900, Sydney–Melbourne 880, Melbourne–Hobart 700, Brisbane–Cairns 1400.

PRIM'S ALGORITHM

PRIM

1. Choose an arbitrary starting vertex.
2. In each round, select the minimum-weight edge in the processed–unprocessed cut.
3. Mark the unprocessed endpoint as processed.

Invariant: at every step we have an MST on the processed set S ; initially S is a single vertex, and $|S|$ grows by one each round.

PRIM'S ALGORITHM

Correctness is immediate: every edge we add is a minimum-weight edge of the cut defined by S , so it belongs to the MST by the Fact.

Contrast with Kruskal: Kruskal maintains a *forest* spread over the whole graph; Prim maintains a single connected tree.

PRIM = DIJKSTRA WITH BOTTLENECK DISTANCE

PRIM

1. Choose an arbitrary starting vertex.
2. Each round: select the minimum-weight edge in the processed-unprocessed cut.
3. Mark the new endpoint processed.

LLSP DIJKSTRA

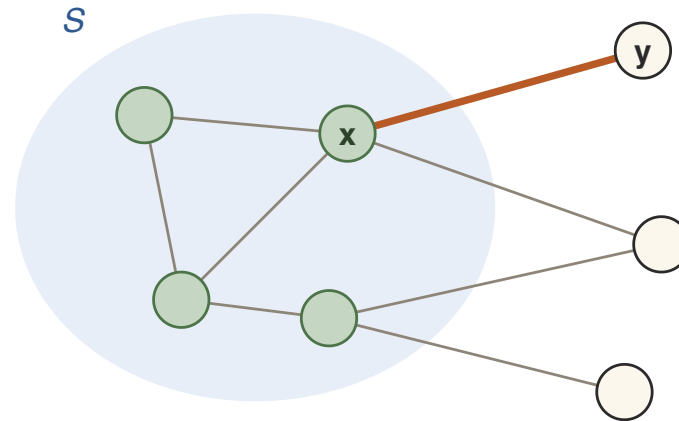
1. Start with the source vertex.
2. Each round: select the unprocessed vertex with minimum `dist_to` value.
3. Relax its outgoing edges, mark it processed.

PRIM = DIJKSTRA WITH BOTTLENECK DISTANCE

CLAIM Prim's algorithm and Dijkstra's algorithm for longest-leg shortest paths can process vertices in exactly the same order.

Proof by induction. Base case: start both at the same vertex. Suppose both have processed the same first k vertices, forming the set S .

SAME NEXT VERTEX: THE CLAIM



(x, y) : the smallest-weight edge in the cut defined by S — Prim's choice.

CLAIM $\text{dist_to}[y] \leq \text{dist_to}[v]$ for any $v \notin S$ — so LLSP Dijkstra may also process y next.

Two cases, depending on how $w(x, y)$ compares with the bottleneck distance $d(0, x)$.

CASE 1: $w(x, y) \geq d(0, x)$

Then the path to x followed by edge (x, y) has bottleneck $w(x, y)$, and

$$d(0, y) = \text{dist_to}[y] = w(x, y).$$

Any path from the source to any $v \notin S$ must contain an edge of the cut defined by S , and every cut edge weighs at least $w(x, y)$.

So every $v \notin S$ has $\text{dist_to}[v] \geq w(x, y) = \text{dist_to}[y]$. ✓

CASE 2: $w(x, y) < d(0, x)$

Then appending the cheap edge (x, y) does not raise the bottleneck: $d(0, y) = d(0, x)$.

For any unprocessed v we have $\text{dist_to}[v] \geq d(0, x)$ — otherwise v would have been processed *before* x (Dijkstra processes bottleneck distances in nondecreasing order).

So again $\text{dist_to}[v] \geq d(0, x) = \text{dist_to}[y]$. ✓

CONCLUSION Prim = LLSP Dijkstra. Growing an MST and computing bottleneck distances are the *same computation*.

PRIM VS. DIJKSTRA: HISTORY

- Prim published his spanning tree algorithm in **1957**.
- Dijkstra published his shortest-path algorithm in **1959** — "A Note on Two Problems in Connexion with Graphs".
- The *same paper* also gave an MST algorithm identical to Prim's — his "Problem 1"; shortest paths were "Problem 2".
- In fact, Dijkstra could have given a single algorithm in a more abstract setting: his shortest-path algorithm run with the bottleneck distance *is* Prim's algorithm.

Jarník found it first, in 1930 — hence “Jarník–Prim–Dijkstra algorithm”.

A LAZY PRIM IMPLEMENTATION

Each round we need the minimum-weight edge in the cut defined by S . Keep a minimum priority queue of edges .

- When a vertex joins S , push all its incident edges that leave S onto the queue.
- To find the next tree edge, pop the top of the queue.
- The popped edge may be stale — pushed long ago, and by now *both* endpoints are in S (adding it would create a cycle). If so, discard it and pop again.

We do not eagerly delete edges the moment they become irrelevant — that is what makes the implementation "lazy".

LAZY PRIM: CODE

```
// marked[v]: is v in S?   edge_queue: min-PQ of edges by weight
// start: mark source, push its incident edges
while (!edge_queue.empty()) {
    Edge min_wt_edge = edge_queue.top();
    edge_queue.pop();
    if (marked[min_wt_edge.v1] != marked[min_wt_edge.v2]) {
        mst.add_edge(min_wt_edge);
        // let vertex_to_add be the new endpoint of min_wt_edge
        marked[vertex_to_add] = true;
        // push edges incident to vertex_to_add that cross
        // the current cut onto edge_queue
    }
}
```

The `if` test throws away stale edges: an edge with both endpoints marked can no longer be in the cut.

PRIM: RUNNING TIME

Analyse the lazy implementation in the adjacency-list model.

- **Pushes:** we only push edges crossing the current cut, so each edge is pushed at most once — at most $|E|$ pushes.
- **Pops:** the while loop runs at most $|E|$ times; each push/pop costs $O(\log |E|)$.
- **Scanning:** the incident edges of `vertex_to_add` are enumerated in $O(\deg(\text{vertex_to_add}))$ total.

PRIM: RUNNING TIME

TOTAL $O(|E| \log |E|)$, and since $|E| < n^2$ this is $O(|E| \log n)$ — the same bound as Kruskal.

TWO MST INVARIANTS

PRIM INVARIANT One tree on a subset S of vertices; all tree edges are in the MST.

KRUSKAL INVARIANT A partition $S_1, \dots, S_k$ of the vertices with a tree on each part; all tree edges are in the MST.

Is there an algorithm where *every component* grabs its cheapest edge at once? Yes — and it is the oldest one of all.

BORŮVKA'S ALGORITHM

THE FIRST MST ALGORITHM

Our third algorithm to compute a minimum spanning tree:
Borůvka's algorithm.

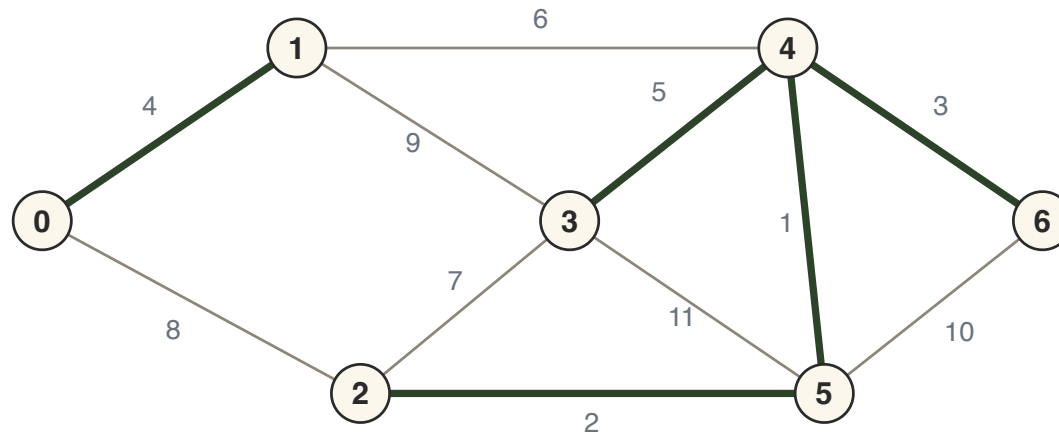
- It was actually the *first* MST algorithm invented — published in **1926**, for planning an electrical network in Moravia.
- In some sense it is the simplest MST algorithm.

THE FIRST MST ALGORITHM

BORŮVKA Proceed in rounds, maintaining a forest of MST edges (initially empty). In each round, *every* connected component simultaneously adds the minimum-weight edge leaving it.

ROUND 1

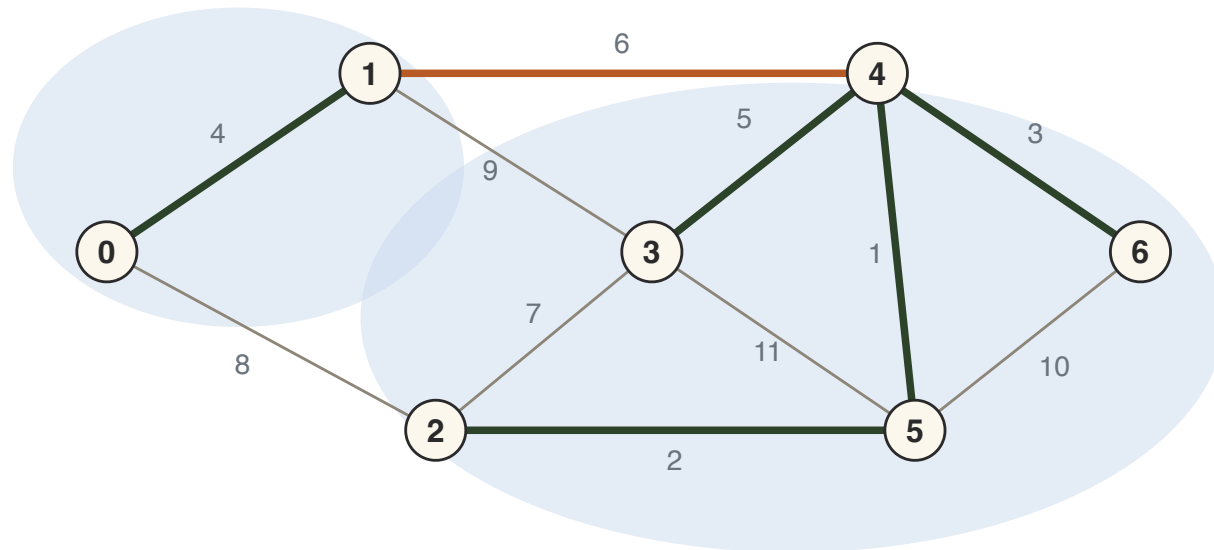
The forest begins empty: each vertex is its own component. Every vertex selects the minimum-weight edge incident to it.



$0, 1 \rightarrow \{0, 1\}$ (weight 4); $2 \rightarrow \{2, 5\}$ (2); $3 \rightarrow \{3, 4\}$ (5); $4, 5 \rightarrow \{4, 5\}$ (1); $6 \rightarrow \{4, 6\}$ (3). All these edges are in the MST — add them all at once. End of Round 1.

ROUND 2

The forest now has two connected components: $\{0, 1\}$ and $\{2, 3, 4, 5, 6\}$. Each chooses the minimum-weight edge in the cut it defines.



Cut edges: $\{0, 2\}$ (8), $\{1, 3\}$ (9), $\{1, 4\}$ (6). Both components pick $\{1, 4\}$ — add it. One component remains, so the algorithm terminates with the MST (weight 21).

BORŮVKA: CORRECTNESS

Every edge added is a minimum-weight edge of the cut around a connected component.

CORRECTNESS Follows immediately from the Fact: the minimum spanning tree must contain the minimum-weight edge in every cut.

CAREFUL As with Kruskal and Prim, with *distinct* edge weights this is airtight; with ties, fix a consistent tie-breaking order (e.g. by edge index) so components agree on their choices.

A GENERAL ROUND

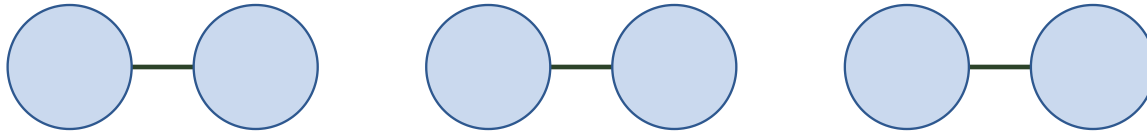
Adding an edge (that creates no cycle) to a forest reduces the number of connected components by one.

Suppose a round begins with T components. Each selects an edge, but selections can coincide: an edge can be selected *at most twice*, once by the component of each endpoint.

So at least $\lceil T/2 \rceil$ distinct edges are added, and at the end of the round at most $T - \lceil T/2 \rceil = \lfloor T/2 \rfloor$ components remain.

AT MOST $\log n$ ROUNDS

Each round (at worst) halves the number of connected components.



worst case: components pair up

ROUNDS Starting from n components, Borůvka's algorithm terminates after at most $\log n$ rounds.

IMPLEMENTING A ROUND

The task in a round: identify the minimum-weight edge leaving each connected component.

Idea 1: union-find. Maintain components in a union-find structure; for each edge, two `find` calls check whether its endpoints lie in distinct components, and if so it competes to be the lightest edge of either component. Afterwards, `union` merges along the chosen edges.

Cost: $O(|E| \log n)$ per round with Week 5's union-find, so $O(|E|(\log n)^2)$ overall. Can we do better?

IMPLEMENTING A ROUND

Idea 2: take time to simplify. We iterate over all edges anyway — so first run your favourite search (BFS/DFS) on the forest edges to label the components $1, 2, 3, \dots$ in time $O(|E|)$, storing each vertex's label in an array.

Now "find" is a constant-time array lookup, and one $O(|E|)$ sweep over the edges finds every component's minimum leaving edge.

BORŮVKA: FINAL COMPLEXITY

$O(|E|)$ work per round, at most $\log n$ rounds:

THEOREM Borůvka's algorithm computes a minimum spanning tree in time $O(|E| \log n)$.

BORŮVKA: FINAL COMPLEXITY

- No fancy data structures required — no priority queue, no union-find.
- Naturally parallel: each component can find the minimum-weight edge leaving it independently.
- Same $O(|E| \log n)$ bound as Kruskal and Prim, all based on one cut principle.

SUMMARY

ALGORITHM	STRATEGY	TIME
Kruskal (W5)	globally lightest edge; union-find	$O(E \log n)$
Prim	grow one tree; lightest cut edge	$O(E \log n)$
Borůvka	lightest edge per component, in rounds	$O(E \log n)$

SUMMARY

- All three are certified by: *the MST contains a min-weight edge of every cut.*
- Prim is exactly Dijkstra run with the bottleneck (longest-leg) distance — and the MST answers all minimax path queries.
- Dijkstra sorts distances as it goes; that sorting barrier was broken for SSSP only in 2025.